SavoryStay — Technical Deep-Dive & Interview Preparation

> *A comprehensive technical guide covering architecture, system design patterns, detailed flows, database design, security model, and key decisions made during the implementation of the SavoryStay restaurant operations platform.*

Table of Contents

---

1. System Overview & Tech Stack

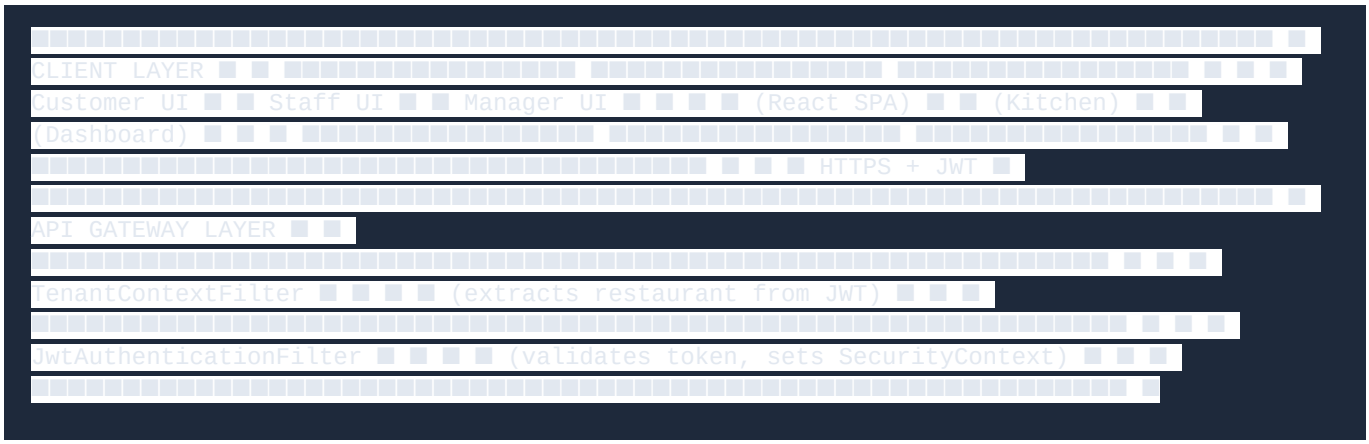
What is SavoryStay?

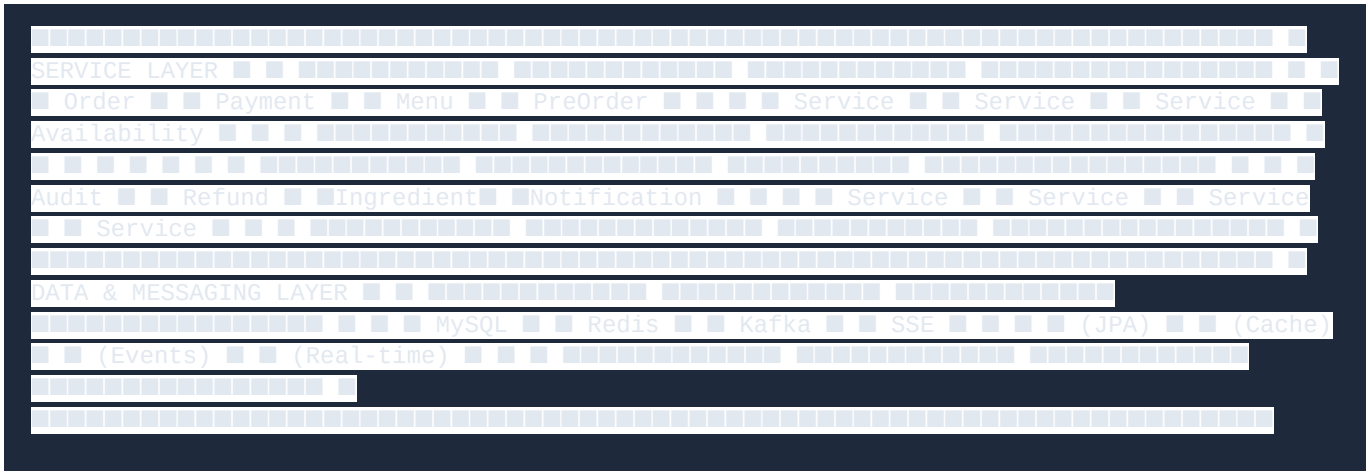
SavoryStay is a **multi-tenant restaurant operations platform** that handles the complete lifecycle from customer ordering through kitchen preparation to payment and completion. It supports multiple restaurants under a single deployment, each with isolated data, staff, menus, and customers.

Tech Stack

Layer	Technology	Why Chosen
Frontend	React 19, TypeScript, Vite, Tailwind CSS	Type safety, fast builds, utility-first CSS
Backend	Spring Boot 3.2, Java 17+	Enterprise-grade security, mature ecosystem, JPA
Database	MySQL 8	ACID compliance, mature, good JSON support
Message Queue	Apache Kafka	Distributed, durable, event sourcing
Cache	Redis	In-memory, pub/sub, TTL support
Security	Spring Security 6.2, JWT	Stateless auth, role-based access
Real-Time	Server-Sent Events (SSE)	One-way push, simpler than WebSocket
Payments	Stripe SDK, PayPal SDK	Industry standard, PCI compliance
SMS	Twilio SDK	Reliable OTP delivery
Email	Spring Mail (Gmail SMTP)	OTP delivery, notifications
Testing	JUnit 5, Mockito, Testcontainers	Unit, integration, and containerized DB tests

Architecture Diagram (Logical)





---

2. Architecture & Design Patterns

2.1 Layered Architecture

The application follows a classic **3-tier layered architecture**:

```
Controller (REST API) → Service (Business Logic) → Repository (Data Access)
```

Each layer has a single responsibility: - **Controllers**: HTTP request/response handling, input validation, role checking - **Services**: Business rules, state transitions, orchestration - **Repositories**: Database queries via Spring Data JPA

2.2 Key Design Patterns

Pattern	Where Used	Purpose
State Machine	`OrderStateMachine`	Enforce valid order status transitions
Repository Pattern	Spring Data JPA repos	Abstract data access, enable testing
Strategy Pattern	Payment gateway selection	Stripe vs PayPal vs Cash processing
Template Method	`ChannelDeliveryService`	SMS, Email, WhatsApp delivery channels
Observer Pattern	Kafka consumers	Decouple order events from notifications
Transactional Outbox	`OutboxService` + `OutboxPoller`	Reliable event publishing

CQRS (Light)	Separate read/write for dashboard	Optimized queries for analytics
Circuit Breaker Concept	Idempotency keys	Prevent duplicate payments/events
Audit Trail	`AuditService`	Append-only record of all mutations
Multi-Tenancy	`TenantContext` + `TenantContextFilter`	Data isolation per restaurant

2.3 Design Decisions & Trade-offs

Q: Why Spring Boot over Node.js? - Spring Security provides enterprise-grade RBAC - JPA/Hibernate for complex relational queries - Transaction management with `@Transactional` - Strong typing prevents runtime errors

Q: Why Kafka over RabbitMQ? - Durable message retention (replay capability) - Partitioned topics for scalability - Consumer groups for parallel processing - Built-in dead-letter topic support

Q: Why SSE over WebSocket? - Customer order updates are one-way (server → client) - Simpler implementation, automatic reconnection - Works through HTTP proxies/CDNs - WebSocket needed only for bidirectional chat (not a requirement)

Q: Why MySQL over PostgreSQL? - Team familiarity - Excellent JSON support for flexible data - Mature replication and tooling - Sufficient for the expected load profile

---

3. Multi-Tenancy Architecture

3.1 Tenant Isolation Strategy

SavoryStay uses **shared database, shared schema** multi-tenancy with a `restaurant_id` column on every tenant-scoped table.

3.2 How Isolation is Enforced

Every repository query filters by `restaurant_id`:

```
// IngredientRepository List<Ingredient> findByRestaurantIdAndActiveTrue(String restaurantId); // MenuItemRepository List<MenuItem> findByRestaurantIdAndStatus(String restaurantId, String status); // OrderRepository List<Order> findByRestaurantIdAndOrderStatus(String restaurantId, String status);
```

Critical rule: The `restaurantId` is NEVER taken from the request body. It always comes from `TenantContext.getCurrentRestaurant()`, which is derived from the JWT.

3.3 Cross-Tenant Access Prevention

```
// In OrderService public Order getOrder(String orderId) { Order order = orderRepository.findById(orderId) .orElseThrow(() -> new RuntimeException("Order not found")); if (!order.getRestaurantId().equals(TenantContext.getCurrentRestaurant())) { throw new SecurityException("Access denied — cross-tenant violation"); } return order; }
```

3.4 Customer Multi-Restaurant Membership

Customers can belong to multiple restaurants:

```
Customer (usr_001) ■■■ Restaurant A (rest_01) — can browse menu, place orders ■■■ Restaurant B (rest_02) — can browse menu, place orders ■■■ Restaurant C (rest_03) — can browse menu, place orders When customer switches restaurant: 1. New JWT issued with updated restaurantId 2. TenantContext switches 3. All subsequent queries scope to new restaurant
```

---

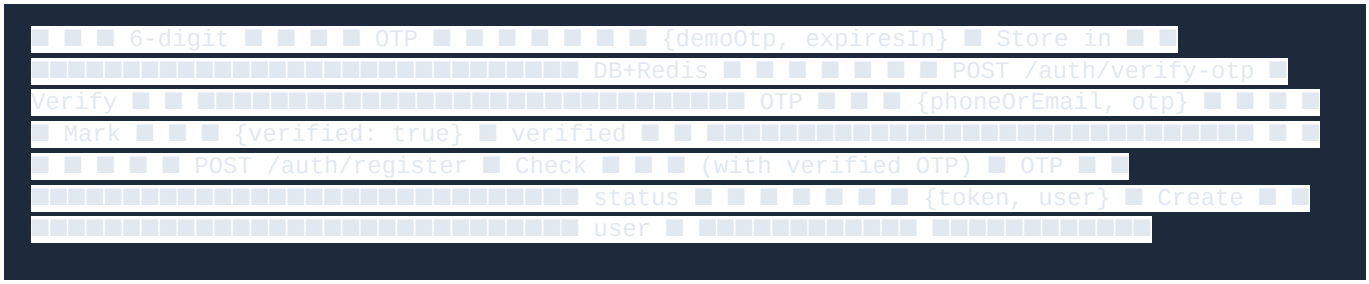
4. Authentication & Authorization

4.1 Authentication Flow

```
■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■ Client ■ ■ Auth ■ ■ Database ■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■ POST /auth/login ■ ■ ■ {email, password} ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ findByEmail() ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ User entity ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ ■ ■ BCrypt.matches() ■ ■ ■ password valid? ■ ■ ■ ■ ■ ■ Generate JWT: ■ ■ ■ - sub: userId ■ ■ ■ - roles: [ROLE_*] ■ ■ ■ - restaurantId ■ ■ ■ - exp: 1 hour ■ ■ ■ ■ ■ {token, user} ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ ■ ■ GET /api/v1/... ■ ■ ■ Authorization: ■ ■ ■ Bearer <jwt> ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ JwtAuthFilter ■ ■ ■ validates token ■ ■ ■ sets SecurityCtx ■ ■ ■ TenantContext ■ ■ ■ ■ ■ 200 OK ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
```

4.2 OTP Flow (Passwordless Login)

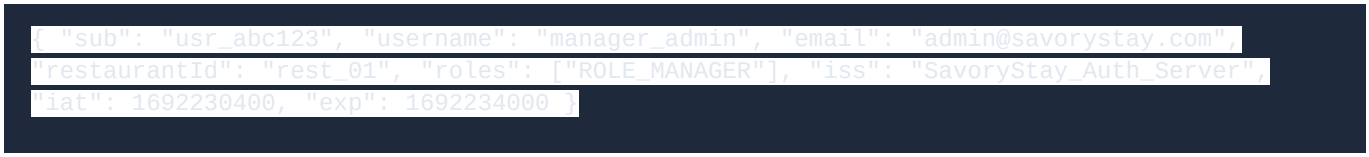
```
■■■■■■■■■■■■■■ POST /auth/send-otp ■■■■■■■■■■■■■■ ■ Client ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Server ■ ■ ■ {phoneOrEmail} ■ ■ ■ ■ ■ Generate ■
```



4.3 Role-Based Authorization

Role	Permissions
CUSTOMER	Browse menu, place orders, track orders, cancel (NEW only)
CHEF	Update order status (PREPARING, PACKED_READY), view kitchen queue
MANAGER	Full menu/order/inventory management, refunds, dashboard
ADMIN	Staff management, customer membership, system config
SUPER_ADMIN	Chain-wide access with explicit restaurant scope

4.4 JWT Token Structure



- **Access Token**: 1 hour lifetime
- **Refresh Token**: 30 days (with token family for revocation)
- **Signing**: HMAC-SHA256

---

5. Order Lifecycle & State Machine

5.1 State Diagram

```
■■■■■■■■■■■■ ■COMPLETED ■ ■CANCELLED ■ ■■■■■■■■■■■■ ■■■■■■■■■■■■
```

5.2 Valid Transitions (OrderStateMachine)

```
public class OrderStateMachine { private static final Map<String, Set<String>> TRANSITIONS = Map.of( "NEW", Set.of("PREPARING", "CANCELLED", "DECLINED"), "PREPARING", Set.of("PACKED_READY", "CANCELLED", "DECLINED"), "PACKED_READY", Set.of("COMPLETED", "CANCELLED"), "COMPLETED", Set.of(), // terminal "CANCELLED", Set.of(), // terminal "DECLINED", Set.of() // terminal ); public static void validate(String from, String to) { if (!TRANSITIONS.getOrDefault(from, Set.of()).contains(to)) { throw new IllegalStateException( "Order cannot transition from " + from + " to " + to); } } }
```

5.3 Role-Based Transition Validation

```
// Chef can only cook and pack if (isChef && !Set.of("PREPARING", "PACKED_READY").contains(newStatus)) { throw new SecurityException("Chef can only set PREPARING or PACKED_READY"); } // Customer can only cancel NEW orders if (isCustomer && !("CANCELLED".equals(newStatus) && "NEW".equals(currentStatus))) { throw new SecurityException("Customers can only cancel NEW orders"); }
```

5.4 Order vs Payment Status (Separated)

```
// These are INDEPENDENT lifecycles Order Status: NEW → PREPARING → PACKED_READY → COMPLETED Payment Status: PENDING → PAID PENDING → FAILED PAID → REFUND_PENDING → REFUNDED // Cash order example: Order: NEW (payment = PENDING) → Staff marks paid at counter (payment = PAID) → Kitchen prepares (order = PREPARING) → Completion (order = COMPLETED)
```

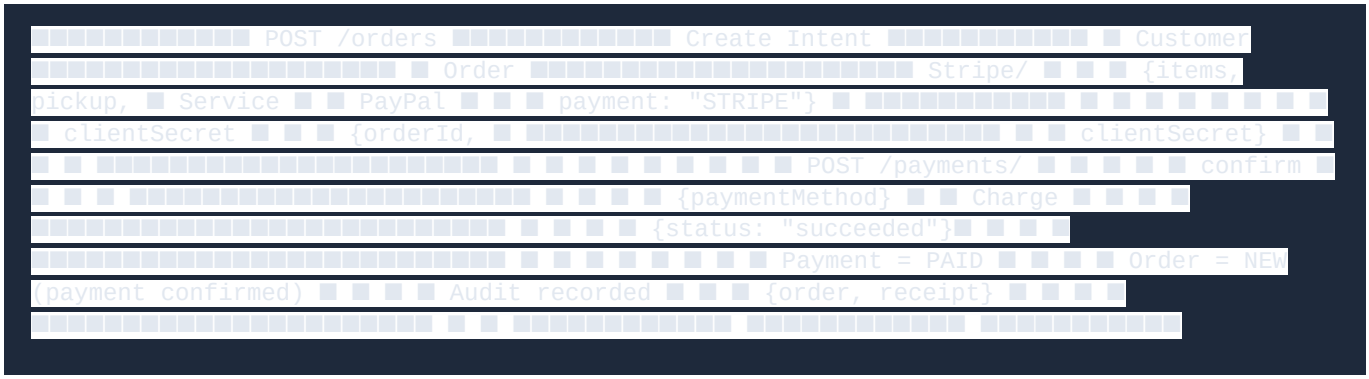
5.5 Idempotency in Order Confirmation

```
public Order confirmPayment(String orderId, String gateway, BigDecimal amount) { Order order = getOrder(orderId); // Idempotent: already paid if ("PAID".equals(order.getPaymentStatus())) { return order; // return existing, don't reprocess } // CASH orders: don't process online if ("CASH".equalsIgnoreCase(gateway)) { throw new IllegalStateException("Cash orders are paid at counter"); } // Amount validation if (amount.compareTo(order.getTotalAmount()) != 0) { throw new IllegalArgumentException("Amount mismatch"); } // Create payment record Payment payment = createPayment(order, gateway, amount); order.setPaymentStatus("PAID"); return order; }
```

---

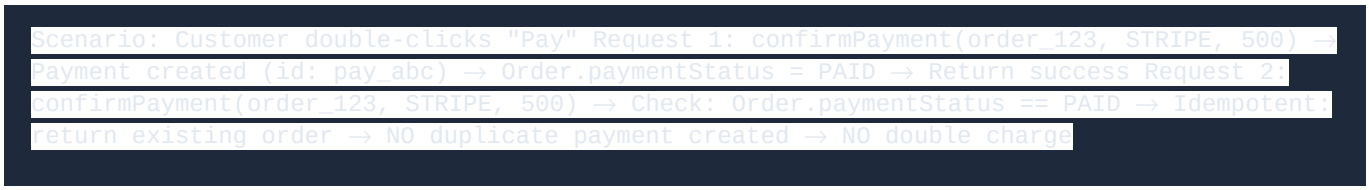
6. Payment System

6.1 Payment Flow — Online Payment

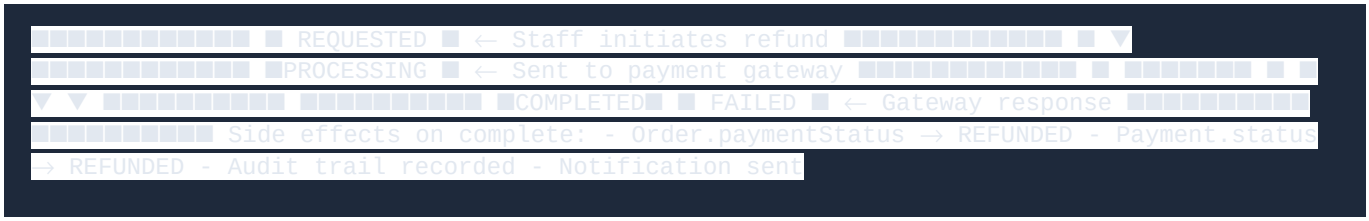


6.2 Payment Flow — Cash (Pay-on-Pickup)

6.3 Payment Idempotency



6.4 Refund Lifecycle



---

7. Pre-Order Availability Engine

7.1 Pre-Order Rules

The system enforces several rules to determine if a dish can be pre-ordered:

```
Can the customer order Dish X for Day Y? Rule 1: Is the restaurant open on Day Y? → Check RestaurantOperatingHour for day-of-week → Check holiday closures in PreOrderSettings Rule 2: Is the dish available on Day Y? → Check DishAvailability (weekly schedule) → Check DishSlotOverride (one-off exceptions) Rule 3: Is the order within the cutoff window? → PreOrderSettings has cutoffTime (e.g., 2:00 PM) → Orders placed after cutoff are for next available day Rule 4: Is it within the booking horizon? → PreOrderSettings has maxAdvanceDays (e.g., 7 days) → Cannot order more than 7 days ahead
```

7.2 7-Day Calendar Availability

```
Customer opens pre-order calendar: Mon Tue Wed Thu Fri Sat Sun Biryani ■ ■ ■ ■ ■ ■ ■ Paneer ■ ■ ■ ■ ■ ■ ■ Cake ■ ■ ■ ■ ■ ■ ■ Today: Wednesday Max advance: 7 days Cutoff: 2:00 PM If now = 3:00 PM → Wednesday unavailable, earliest = Thursday If now = 1:00 PM → Wednesday available (if open)
```

7.3 Pre-Order Availability Service Flow

```
public DishAvailabilityResponse checkAvailability( String dishId, LocalDate date, LocalTime time) { // 1. Restaurant open? if (!restaurantService.isOpen(restaurantId, date.getDayOfWeek())) { return unavailable("Restaurant closed"); } // 2. Holiday? if (preOrderConfigService.isHoliday(restaurantId, date)) { return unavailable("Holiday"); } // 3. Dish available on this day? if (!dishAvailabilityService.isAvailable(dishId, date)) { return unavailable("Dish not scheduled"); } // 4. Slot override? Optional<DishSlotOverride> override = dishSlotOverrideRepository.findByDishIdAndDate(dishId, date); if (override.isPresent()) { return override.get().isAvailable() ? available() : unavailable("Override: unavailable"); } // 5. Within cutoff? if (preOrderConfigService.isPastCutoff(restaurantId, date, time)) { return unavailable("Past cutoff time"); } return available(); }
```

---

8. Inventory & Ingredient Management

8.1 Ingredient as Master Data

```
Restaurant ■ ■■■ Ingredient #101 (Rice) ■ ■■■ MenuItemIngredient → Biryani (500g) ■ ■■■ MenuItemIngredient → Pulao (300g) ■ ■■■ InventoryLedger → current stock: 25 kg ■ ■■■ Forecast → aggregated demand ■ ■■■ Ingredient #102 (Chicken) ■ ■■■ MenuItemIngredient → Biryani (250g) ■ ■■■ MenuItemIngredient → Butter Chicken (300g) ■ ■■■ InventoryLedger → current stock: 18 kg ■ ■■■ Ingredient #103 (Ghee) ■■■ ...
```

Key principle: Ingredient ID is the canonical identity, NOT the name.

8.2 Ingredient Name Normalization

```
public class IngredientNormalization { // " Chicken Breast " → "chicken breast" // "RICE" → "rice" // "riCe" → "rice" public static String normalize(String name) { return name.trim() .toLowerCase() .replaceAll("\\s+", " "); } } // Database constraint: // UNIQUE(restaurant_id, normalized_name) // → Prevents duplicates within a restaurant // → Same name allowed across restaurants
```

8.3 Inventory Lifecycle

```
Ingredient: Chicken (stock: 18 kg) Order placed → PREPARING: → Deduct: 18 kg - 0.5 kg = 17.5 kg → Ledger entry: CONSUMPTION, -0.5 kg Order cancelled: → Release: 17.5 kg + 0.5 kg = 18 kg → Ledger entry: CANCELLATION_RELEASE, +0.5 kg Adjustment: → Manual: 18 kg → 20 kg → Ledger entry: PURCHASE, +2 kg → Reason: "Weekly chicken delivery"
```

8.4 Ingredient Forecast Aggregation

```
Tomorrow's pre-orders: 10x Biryani (requires 500g Rice each) 5x Pulao (requires 300g Rice each) 3x Risotto (requires 400g Rice each) Forecast (by ingredient ID): Rice #101: (10 × 500g) + (5 × 300g) + (3 × 400g) = 5000g + 1500g + 1200g = 7.7 kg Current stock: 25 kg After production: 25 - 7.7 = 17.3 kg Shortfall: 0 (sufficient)
```

Critical: Aggregation is by `ingredientId`, NOT by name. This eliminates name-based errors.

---

9. Event-Driven Architecture (Kafka)

9.1 Event Flow

```
[redacted] Publish Event [redacted] Consume Event [redacted] Order [redacted] Kafka [redacted] OTP Service otp.generated Topic otp.generated Consumer order.placed order.placed Order [redacted] Consumer payment.success payment.success Payment [redacted] Consumer inventory.low inventory.low Inventory [redacted] Consumer [redacted]
```

9.2 Kafka Configuration

```
@Configuration public class KafkaTopicConfig { @Bean public NewTopic otpTopic() { return TopicBuilder.name("savorystay.otp") .partitions(3) .replicas(1) .build(); } // Retry topic with backoff // DLT (Dead Letter Topic) for failed messages }
```

9.3 Retry & Dead Letter Topic

```
Event published → savorystay.otp ■ ▼ (consumer fails) savorystay.otp-retry-4000 (4 second delay) ■ ▼ (still fails) savorystay.otp-retry-8000 (8 second delay) ■ ▼ (exhausted retries) savorystay.otp-dlt (Dead Letter Topic) ■ ▼ DltRecorder logs the failure for investigation
```

9.4 Event Publishing (Transactional Outbox)

To ensure events are published **exactly once** even if Kafka is temporarily unavailable:

```
Step 1: Business transaction begins Step 2: Business data written (order, payment, etc.) Step 3: Event written to outbox table (same transaction) Step 4: Transaction commits Step 5: Separate poller reads outbox and publishes to Kafka Step 6: Published event marked as sent in outbox
```

This guarantees: - If the transaction succeeds, the event WILL eventually be published - If the transaction fails, no event is created - Kafka outage doesn't lose events

---

10. Caching Strategy (Redis)

10.1 What is Cached

Cache Key Pattern	TTL	Purpose
`menu:{restaurantId}`	5 min	Menu items for a restaurant
`preorder:avail:{dishId}:{date}`	1 min	Dish availability check
`otp:{phoneOrEmail}`	10 min	OTP verification codes
`ratelimit:{ip}:{endpoint}`	Configurable	Auth rate limiting
`session:{userId}`	1 hour	Active sessions

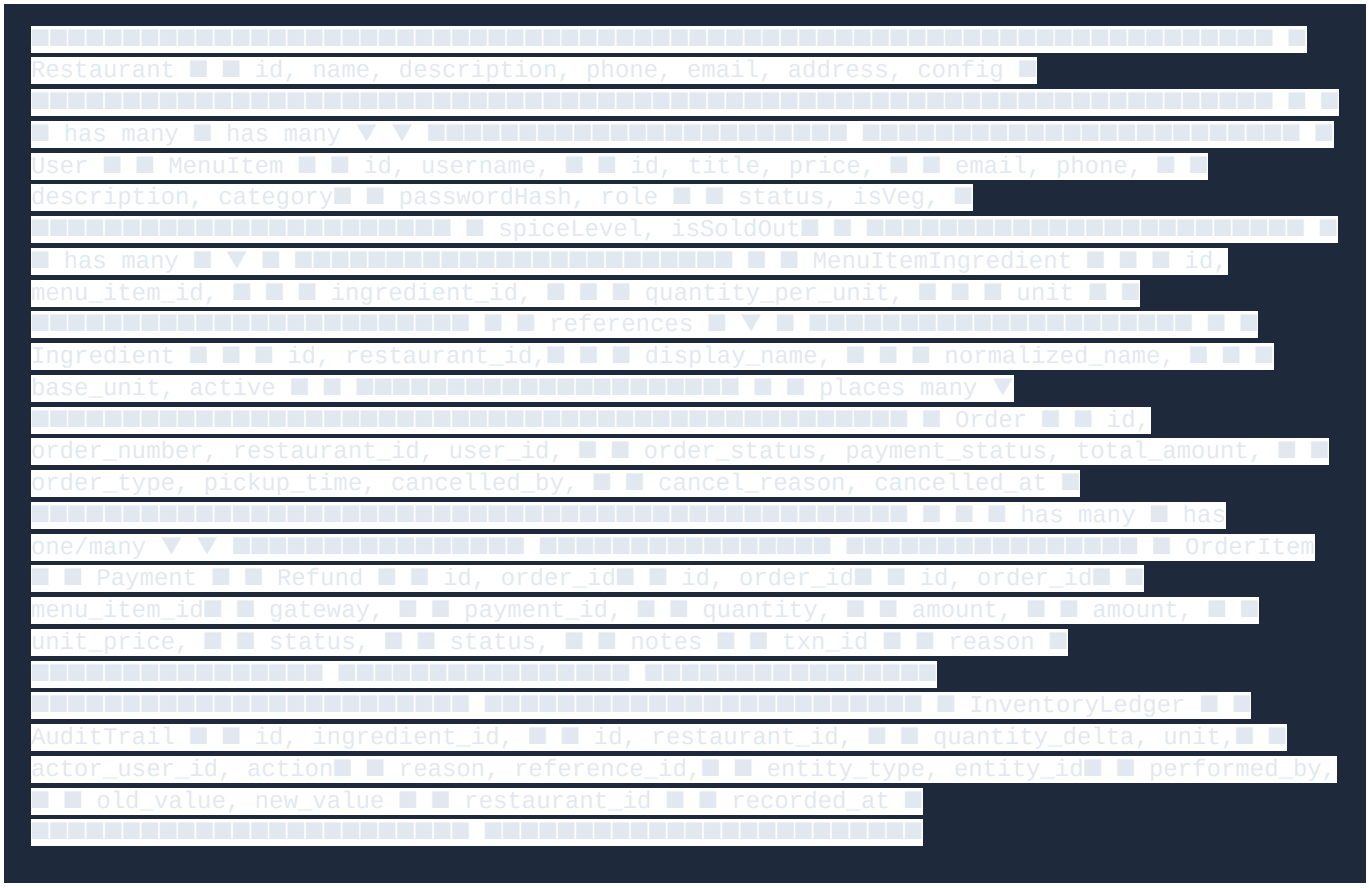
10.2 Cache Invalidation

```
Menu updated by manager → Invalidate cache: menu:{restaurantId} → Next request fetches fresh data from DB → Cache populated again Pre-order configuration changed → Invalidate: preorder:avail:* for affected dishes → Availability recalculated on next request
```

---

11. Database Design & ERD

11.1 Core Entity Relationships



11.2 Key Database Constraints

```
-- Ingredient uniqueness per restaurant ALTER TABLE ingredients ADD CONSTRAINT
uq_ingredient_restaurant_name UNIQUE (restaurant_id, normalized_name); -- No duplicate
ingredients in a recipe ALTER TABLE menu_item_ingredients ADD CONSTRAINT
uq_recipe_ingredient UNIQUE (menu_item_id, ingredient_id); -- Index for fast tenant
queries CREATE INDEX idx_orders_restaurant_status ON orders(restaurant_id, order_status);
CREATE INDEX idx_menu_items_restaurant ON menu_items(restaurant_id, status); CREATE INDEX
idx_ingredients_restaurant_active ON ingredients(restaurant_id, active); -- Audit trail
indexes CREATE INDEX idx_audit_restaurant_recorded ON audit_trail(restaurant_id,
recorded_at DESC); CREATE INDEX idx_audit_entity ON audit_trail(entity_type, entity_id);
```

---

12. Real-Time Updates (SSE)

12.1 SSE Architecture

```
GET /api/v1/sse/orders
Kitchen
SSE
Screen
Connection stays open
Endpoint
Event: new-order
{"orderId":"ord_123",
"dish":"Biryani",
"qty":2}
Event:
```

```
order-updated ■ ■ ■ ■ {"orderId":"ord_123", ■ ■ ■ ■ "status":"PACKED_READY"}■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■ ■■■■■■■■■■■■
```

12.2 SSE vs WebSocket

Aspect	SSE	WebSocket
Direction	Server → Client	Bidirectional
Protocol	HTTP/1.1	Custom (ws://)
Auto-reconnect	Built-in	Manual
Proxy-friendly	Yes (standard HTTP)	Sometimes blocked
Use case	Order updates, notifications	Chat, collaboration
Complexity	Low	Medium

Decision: SSE was chosen because order updates are one-way (server pushes status changes). WebSocket would add unnecessary complexity.

---

13. Transactional Outbox Pattern

13.1 Problem Solved

Without the outbox pattern, you face the **dual-write problem**:

```
// UNRELIABLE (without outbox): @Transactional public Order placeOrder(Order order) {
orderRepo.save(order); // Step 1: DB write ■ kafka.publish("order.placed"); // Step 2:
Kafka write ■ (Kafka down!) } // Order saved but event lost → inconsistency
```

13.2 Solution: Outbox Table

```
// RELIABLE (with outbox): @Transactional public Order placeOrder(Order order) {
orderRepo.save(order); // Step 1: DB write outboxService.record("order.placed", payload);
// Step 2: Outbox write // BOTH succeed or BOTH fail (same transaction) } // Later, a
poller reads unsent outbox events: @Scheduled(fixedDelay = 5000) public void pollOutbox()
{ List<OutboxEvent> unsent = outboxRepo.findBySentFalse(); for (OutboxEvent event :
unsent) { kafka.publish(event.getTopic(), event.getPayload()); event.setSent(true);
outboxRepo.save(event); } }
```

13.3 Guarantees

Scenario	What Happens
DB commit succeeds, Kafka up	Event published immediately
DB commit succeeds, Kafka down	Event stays in outbox, retried by poller
DB commit fails	No event recorded (atomic)
Poller crashes	Next poll picks up unsent events
Kafka accepts but doesn't persist	Outbox stays unsent, retried

---

14. Key Design Patterns Used

14.1 Idempotency

Where: Payment confirmation, webhook handling, order creation, refund initiation.

```
// Idempotent payment confirmation public Payment confirmPayment(String orderId, String gateway, BigDecimal amount) { // Check if already processed Optional<Payment> existing = paymentRepository.findByOrderIdAndStatus(orderId, "PAID"); if (existing.isPresent()) { return existing.get(); // Idempotent: return existing } // ... create new payment }
```

14.2 Optimistic Locking

Where: Inventory deduction to prevent overselling.

```
@Transactional public void consumeInventory(String ingredientId, BigDecimal quantity) { Ingredient ingredient = ingredientRepository.findById(ingredientId) .orElseThrow(); BigDecimal newStock = ingredient.getCurrentStock().subtract(quantity); if (newStock.compareTo(BigDecimal.ZERO) < 0) { throw new InsufficientInventoryException("Not enough stock"); } ingredient.setCurrentStock(newStock); ingredientRepository.save(ingredient); // JPA @Version field prevents concurrent overwrites }
```

14.3 Circuit Breaker Concept

Where: Kafka event publishing with retry and DLT.

```
Normal flow: Publish → Kafka topic ↓ (fails) Retry flow: Publish → retry-4000 → retry-8000 → DLT ↓ DltRecorder logs failure
```

14.4 Strangler Fig Pattern

Where: Migration from Express.js/Firebase to Spring Boot/MySQL.

```
Phase 1: Spring Boot runs alongside Express Phase 2: New features built in Spring Boot
only Phase 3: Express routes migrated one by one Phase 4: Express removed (current state)
```

14.5 CQRS (Light)

Where: Dashboard queries use optimized read models.

```
Write path: Order → OrderService → OrderRepository → MySQL Read path:
DashboardController → Aggregate queries → DTO response Dashboard doesn't load full Order
entities. It uses SQL aggregation: SELECT order_status, COUNT(*) FROM orders WHERE
restaurant_id = ? AND DATE(created_at) = CURDATE() GROUP BY order_status
```

---

15. Testing Strategy

15.1 Test Pyramid

```
■■ ■ ■ ■ E2E■ ← Manual / Selenium (future) ■■■■■■■■ ■ Integration ■ ← 18
Testcontainers tests ■■■■■■■■■■■■■■■■■■■■ ■ Unit Tests ■ ← 234 unit tests
■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

15.2 Test Categories

Category	Count	What's Tested
Unit	234	Services, state machine, DTOs, security
Integration	18	Full Spring context + real MySQL via Testcontai
Total	**252**	

15.3 Key Test Scenarios

```
// State Machine tests @Test void validTransition() // NEW → PREPARING ✓ @Test void invalidTransition() // COMPLETED → PREPARING ✗ @Test void terminalState() // CANCELLED → anything ✗ // Tenant Isolation tests @Test void crossTenantAccess() // REST_A user accesses REST_B → 403 // Payment Idempotency tests @Test void doubleConfirm() // Same order confirmed twice → no duplicate // Refund tests @Test void refundLifecycle() // REQUESTED → PROCESSING → COMPLETED @Test void refundIdempotent() // Same refund initiated twice → returns existing // Inventory tests @Test void concurrentDeduction() // Two orders simultaneously → only one succeeds @Test void cancelReleasesStock() // Cancelled order returns inventory // Integration tests (Testcontainers) @SpringBootTest @Testcontainers class RefundAuditIntegrationTest { @Container static MySQLContainer<?> mysql = new MySQLContainer<>("mysql:8.0"); @Test void fullRefundLifecycle() // Real DB, real transactions @Test void auditTrailQuery() // Complex aggregation queries @Test void paymentIdempotency() // End-to-end idempotency }
```

---

16. Scalability Considerations

16.1 Current Architecture Limits

Component	Bottleneck	Mitigation
MySQL	Single writer	Read replicas (future)
Redis	Single instance	Redis Cluster (future)
Kafka	Single broker	Multi-broker cluster (production)
Spring Boot	Vertical scaling	Horizontal scaling behind LB
SSE	Connection limits	Reverse proxy (Nginx) buffering

16.2 What Would Change at Scale

10x traffic: - Add MySQL read replicas - Increase Kafka partitions - Add Redis Cluster - CDN for frontend static assets

100x traffic: - Consider CQRS with separate read/write databases - Event sourcing for order history - Microservice decomposition (order service, payment service, notification service) - Kubernetes orchestration

16.3 Query Optimization

```
-- Dashboard summary: aggregate query, not application-side loop SELECT order_status, COUNT(*) as count, SUM(total_amount) as revenue FROM orders WHERE restaurant_id = ? AND DATE(created_at) = CURDATE() GROUP BY order_status; -- Ingredient forecast: aggregate by
```

```
ingredient_id, not name SELECT i.id, i.display_name, SUM(mii.quantity_per_unit *
oi.quantity) as total_required FROM order_items oi JOIN menu_item_ingredients mii ON
oi.menu_item_id = mii.menu_item_id JOIN ingredients i ON mii.ingredient_id = i.id WHERE
i.restaurant_id = ? AND i.active = true GROUP BY i.id, i.display_name;
```

---

17. Common Interview Questions & Answers

System Design Questions

Q: How do you handle concurrent orders that might oversell inventory?

> *We use database-level atomic updates with JPA `@Version` optimistic locking. When an order is placed, the inventory deduction is atomic — if two concurrent orders try to consume the same stock, one will succeed and the other will get a `OptimisticLockException`. We catch this and return a user-friendly "insufficient inventory" error. Additionally, the `IngredientService` uses `@Transactional` to ensure the entire deduction is atomic.*

Q: How does the system handle payment failures gracefully?

> *If an online payment fails, the order remains in `NEW` state with `PENDING` payment status. The customer can retry. We use idempotency keys to prevent double-charging. For webhook failures, the transactional outbox pattern ensures events are eventually published. Dead letter topics capture events that fail after all retries.*

Q: How do you ensure tenant isolation in a shared database?

> *Every request goes through `TenantContextFilter` which extracts `restaurantId` from the JWT token. This is stored in a `ThreadLocal` via `TenantContext`. Every repository query filters by `restaurantId`. We NEVER trust the client-provided `restaurantId` — it always comes from the authenticated token. Cross-tenant access attempts return 403/400.*

Q: Explain the transactional outbox pattern you used.

> *The problem: if we save an order and then publish to Kafka, and Kafka is down, we lose the event. The solution: we write the event to an `outbox_event` table in the same database transaction as the order. A separate `OutboxPoller` (scheduled task) reads unsent events and publishes them to Kafka. This guarantees exactly-once delivery semantics without distributed transactions.*

Q: How would you design the pre-order availability system?

> *The system checks 5 rules in sequence: (1) Is the restaurant open? (2) Is it a holiday? (3) Is the dish scheduled for this day? (4) Are there one-off overrides? (5) Is it within the cutoff time? Each rule can independently reject the pre-order. The results are cached in Redis with a 1-minute TTL for performance.*

Behavioral Questions

Q: What was the most challenging part of this project?

> *The most challenging part was implementing the order-payment-inventory consistency model. Getting the transaction boundaries right — when to reserve inventory, when to consume it, what happens on cancellation, and how refunds interact — required careful analysis of every business event. The integration tests with Testcontainers were crucial in catching bugs like the refund idempotency issue.*

Q: How did you approach the migration from Express/Firebase to Spring Boot/MySQL?

> *We used the Strangler Fig pattern. New features were built exclusively in Spring Boot. Existing functionality was migrated route by route. Firebase data was backed up and migrated to MySQL. The frontend was updated to call Spring Boot endpoints instead of Express. At no point was the application down during migration.*

Q: How did you ensure code quality?

> *We followed a rigorous testing strategy: 234 unit tests for business logic, 18 integration tests with real MySQL via Testcontainers for critical flows. We used `@PreAuthorize` for authorization, `TenantContext` for isolation, and comprehensive error handling. The `GlobalExceptionHandler` ensures consistent error responses without exposing internals.*

Appendix A: API Endpoint Summary

Auth

Method	Endpoint	Description
POST	`/auth/register`	Register with OTP verification
POST	`/auth/login`	Login with email/password
POST	`/auth/login-otp`	Login with OTP
POST	`/auth/send-otp`	Send OTP to phone/email
POST	`/auth/verify-otp`	Verify OTP code
POST	`/auth/refresh`	Refresh JWT token
GET	`/auth/me`	Current user profile

Orders

Method	Endpoint	Description

POST	`/orders`	Place order
GET	`/orders`	List orders (filtered by role)
GET	`/orders/{id}`	Get order details
PUT	`/orders/{id}/status`	Update order status (state machine)
POST	`/orders/{id}/cancel`	Cancel order
POST	`/orders/{id}/refund`	Initiate refund
POST	`/orders/{id}/confirm-payment`	Confirm payment
POST	`/orders/{id}/items/{itemId}/notes`	Update kitchen notes

Kitchen

Method	Endpoint	Description
GET	`/orders/kitchen/production`	Today's production view
GET	`/orders/kitchen/delayed`	Delayed orders list

Menu

Method	Endpoint	Description
GET	`/menu`	List menu items
POST	`/menu`	Create menu item
PUT	`/menu/{id}`	Update menu item
DELETE	`/menu/{id}`	Delete menu item
POST	`/menu/{id}/sold-out`	Toggle sold-out status

Ingredients

Method	Endpoint	Description
GET	`/ingredients`	List ingredients
POST	`/ingredients`	Create ingredient

PUT	`/ingredients/{id}`	Update ingredient
PATCH	`/ingredients/{id}/status`	Activate/deactivate
GET	`/ingredients/forecast`	Ingredient forecast

Dashboard

Method	Endpoint	Description
GET	`/dashboard/summary`	Today's operational summary
GET	`/dashboard/exceptions`	Operational exceptions
GET	`/dashboard/shopping-list`	Shortfall ingredients
GET	`/dashboard/cash-reconciliation`	Cash reconciliation
GET	`/dashboard/payment-reconciliation`	Payment reconciliation
GET	`/dashboard/tomorrow-brief`	Tomorrow's operations brief

Pre-Order

Method	Endpoint	Description
GET	`/preorder/availability/{dishId}`	Check dish availability
GET	`/preorder/calendar`	7-day availability calendar
PUT	`/preorder/settings`	Update pre-order config

Appendix B: Technology Concepts Quick Reference

Concept	What It Is	Where Used
BCrypt	Password hashing algorithm	User passwords
JWT	JSON Web Token for stateless auth	All authenticated requests
JPA/Hibernate	ORM for database access	All entity persistence

@Transactional	Declarative transaction management	Service layer methods
Optimistic Locking	@Version field prevents lost updates	Inventory deduction
Dead Letter Topic	Queue for undeliverable messages	Failed Kafka events
CQRS	Separate read/write models	Dashboard queries
Transactional Outbox	Reliable event publishing	Kafka event delivery
SSE	Server-Sent Events for real-time push	Order status updates
Multi-Tenancy	Data isolation per restaurant	All data access
RBAC	Role-Based Access Control	Authorization
Testcontainers	Docker-based integration testing	MySQL integration tests
Idempotency	Same operation produces same result	Payment, webhooks
Circuit Breaker	Fail gracefully when dependency is down	Kafka retries + DLT

Document generated for SavoryStay — Technical Interview Preparation Last updated: August 2026